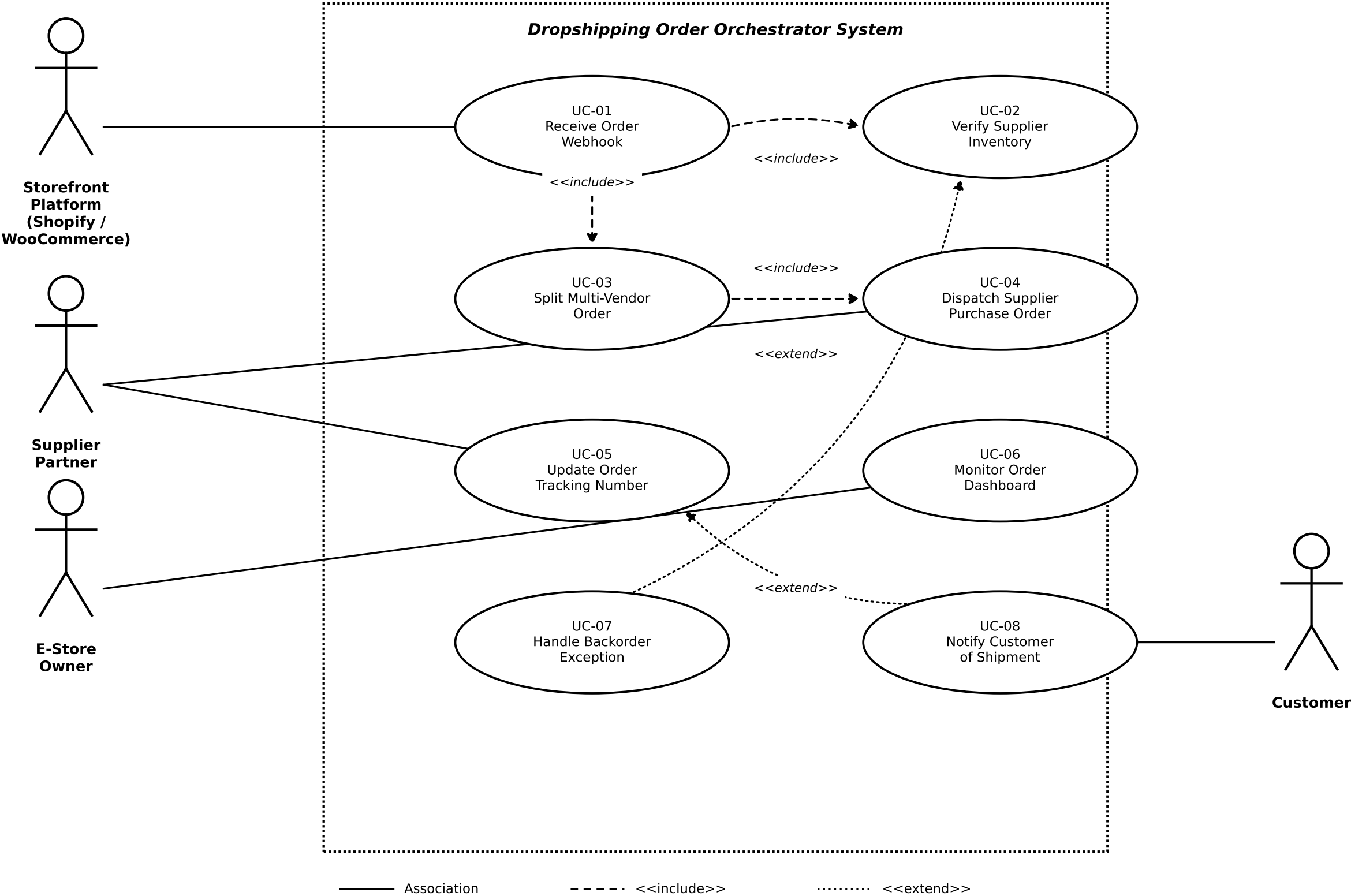


Requirements Table

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001*	Functional	System parses incoming Shopify/WooCommerce order webhooks, checks supplier stock through their API, and sends out a purchase order automatically.	High	Pass: supplier PO is sent with the customer's delivery address attached. Fail: an order goes through for an SKU that's actually out of stock.	This is the core function — without it nothing else in the system runs.
FR-002	Functional	System splits an order that has items from multiple suppliers into separate sub-orders, one per supplier, before any PO is created.	High	Pass: an order with items from 3 suppliers creates exactly 3 sub-orders with the right SKUs in each. Fail: an SKU ends up in the wrong sub-order, or is duplicated/missing.	Each supplier should only see and get billed for their own items.
FR-003	Functional	System records the tracking number a supplier submits and links it to the matching customer order.	High	Pass: tracking number shows up on the customer's order within 5 minutes of entry. Fail: it's missing after 5 minutes, or attached to the wrong order.	Customers shouldn't have to email support just to know where their package is.
FR-004	Functional	E-Store Owner can view a dashboard showing the live status of every order (Received, PO Sent, Shipped, Delivered, Backordered) across all suppliers.	Medium	Pass: dashboard reflects a status change within 2 minutes. Fail: status is stale beyond 2 minutes or doesn't match the actual order state.	Gives the owner one place to check instead of logging into every supplier portal.
FR-005	Functional	If a supplier reports low or no stock for an SKU, the order is automatically put on backorder hold and the E-Store Owner is notified.	High	Pass: order status changes to Backordered and owner gets an alert within 1 minute. Fail: the PO still goes out despite no stock, or no alert is sent.	Stops the store from selling something it can't actually deliver.
NFR-001*	Non-functional	Webhook processing has to acknowledge receipt with HTTP 200 within 200 ms, so the storefront platform doesn't retry and flood the system. (Performance & Security)	High	Pass: load testing confirms the 200 ms response time and expected security checks under peak traffic.	Slow acknowledgements cause storefronts to retry the webhook, which can duplicate orders.
NFR-002	Non-functional	System should keep 99.9% uptime for webhook and supplier polling, and retry a failed supplier API call up to 3 times (with backoff) before giving up. (Reliability & Scalability)	High	Pass: monthly uptime logs show at least 99.9%, and failed calls are retried 3 times before an alert fires. Fail: uptime drops below that, or a failure gets flagged without retrying first.	Supplier APIs go down sometimes — the system needs to handle that instead of just losing the order.

* given in the problem statement.

Use-Case Diagram: Dropshipping Inventory & Order Orchestrator



Use-Case Flow Specification

Use Case Name: Process Incoming Order & Dispatch Supplier Purchase Order(s)

Use Case ID: UC-01/03/04 (Composite)	Actors: Storefront Platform (primary trigger), Supplier Partner, E-Store Owner
---	---

Preconditions

- Webhook endpoint is already registered with the Shopify/WooCommerce store.
- Supplier API credentials are valid for every supplier used in the order.
- Product-to-supplier SKU mapping is up to date.

Postconditions (Success)

- A purchase order has been sent to each relevant supplier, including the customer's delivery address.
- Order status is updated to "PO Dispatched" and shows up on the owner's dashboard.
- The transaction is logged.

Main Success Scenario

1. Storefront platform sends an order-created webhook to the system.
2. System acknowledges the webhook with HTTP 200 within 200 ms (NFR-001).
3. System reads the payload and pulls out the SKUs, quantities, and delivery address.
4. System checks stock for each SKU with the relevant supplier API (UC-02).
5. All suppliers confirm enough stock is available.
6. System splits the order into one sub-order per supplier (UC-03).
7. System sends a purchase order to each supplier with the delivery address attached (UC-04).
8. Order status changes to "PO Dispatched" and the action is logged.
9. Owner sees the updated status on the dashboard (UC-06).
10. Use case ends.

Alternate Flow A — Supplier is out of stock (extends UC-02 via UC-07)

- 5a. A supplier API reports that an SKU is out of stock.
- 5a.1. That sub-order is marked "Backordered" and left out of the PO batch.
 - 5a.2. Owner gets notified with the SKU, quantity, and supplier (FR-005).
 - 5a.3. Owner either finds another supplier or cancels that item.
 - 5a.4. Flow picks back up at step 6 for the rest of the order.

Alternate Flow B — Supplier API times out

- 4a. A supplier API call doesn't respond in time.
- 4a.1. System retries with backoff, up to 3 times (NFR-002).
 - 4a.2. If it still fails, that sub-order is marked "Supplier Unreachable" and the owner is alerted.
 - 4a.3. Flow resumes at step 6 once the supplier is reachable again or the owner steps in manually.